C++ Basics

FDE Tutorial Sessions, Week 1

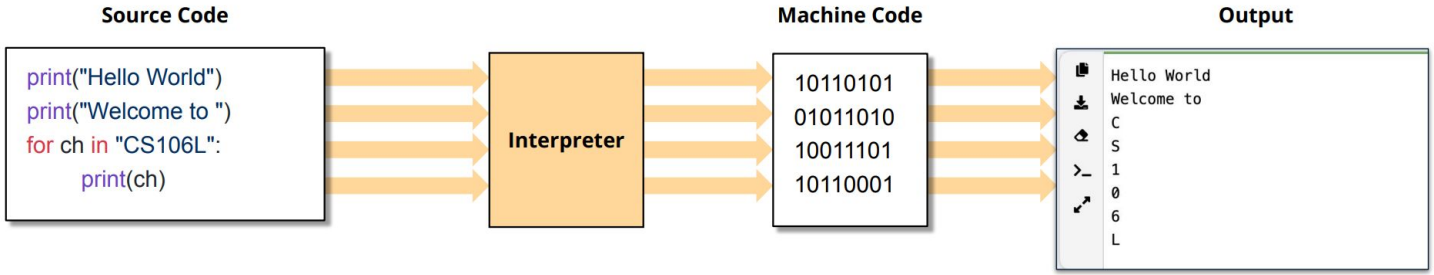
Disclaimer

These slides are supplementary materials provided by the tutor and are not endorsed by the teaching team. They are intended to summarize the key concepts, provide pointers to additional resources and make the tutorial sessions easier to follow for students who are unfamiliar with the topic.

Please note that, in the event of any differences, the official lectures, solution sheets and other materials uploaded to Moodle should take precedence, particularly when preparing for exams.

Toolchain Setup

Interpreted Languages



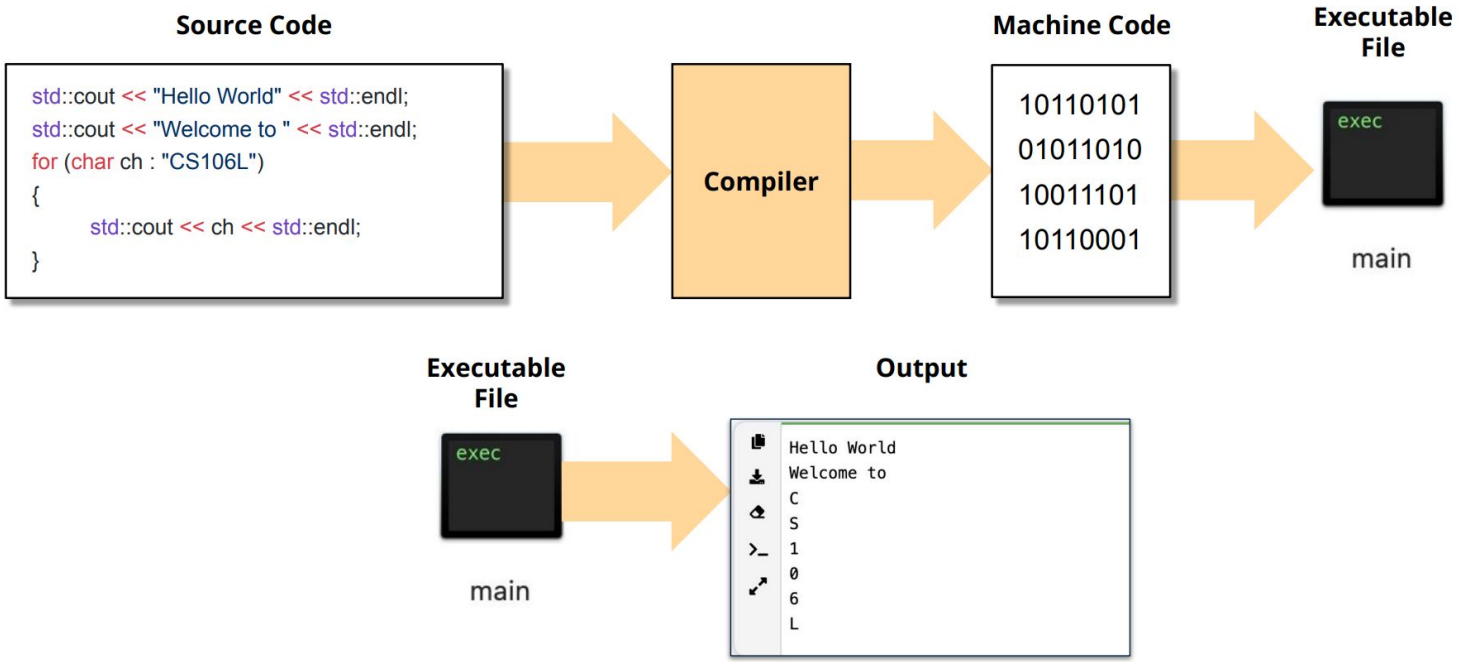
THE BIG IDEA

The interpreted languages read each line of code

line-by-line, **translate** each line, and then **execute** it

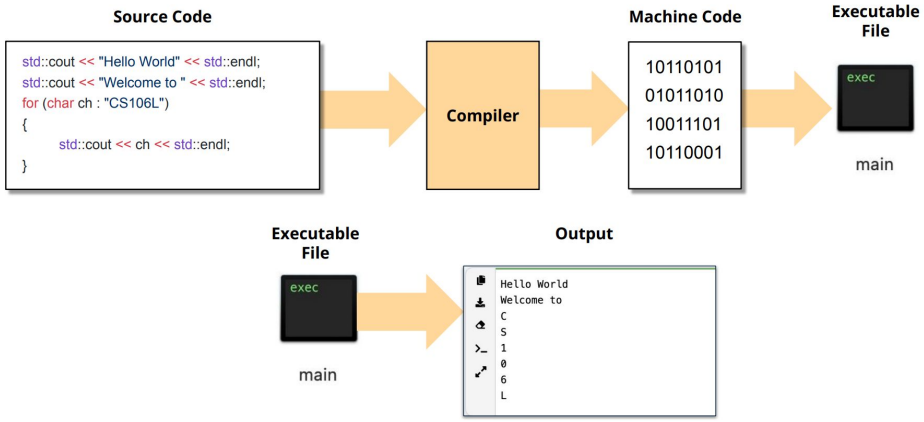
Img src: https://web.stanford.edu/class/cs106l/ (2. Types & Structs)

Compiled Languages



Img src: https://web.stanford.edu/class/cs106l/ (2. Types & Structs)

Compiled Languages



THE BIG IDEA

The compiler translates the **ENTIRE** program, packages it into an **executable file**, and then (vvv the user) **executes** it

Img src: https://web.stanford.edu/class/cs106l/ (2. Types & Structs);
More information on translation: https://cppreference.com/cpp/language/translation_phases

Trade-off

Write **efficient** (resource optimization)

but

human-readable (maintainability)

code!

Build Types

Debug

- Goal is to find bugs
- Additional debug information
- Code is not optimized

Release

- Goal is fast execution
- Code is optimized
- Asserts are disabled

Src: https://stackoverflow.com/a/59314670

Build Types

RelWithDebInfo

- Optimized
- Debug info is included
- Asserts are disabled
- Good for profiling (=examining app for potential performance improvements)

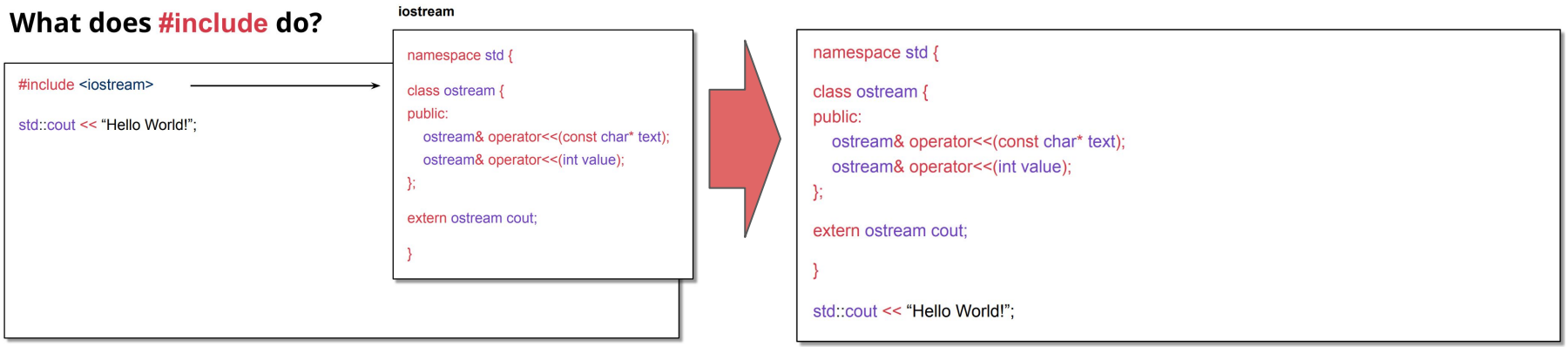
Src: https://stackoverflow.com/a/59314670

Attendance sheet

Headers and Source Files

Headers

“The entities in the C++ standard library are defined in headers, whose contents are made available to a translation unit when it contains the appropriate **`#include`** preprocessing directive.” (C++ Reference)



Src: https://en.cppreference.com/cpp/standard_library;
Img src: https://web.stanford.edu/class/cs106l/ (2. Types & Structs)

Double includes

File "Grandparent.h" [edit]

```
struct Foo {
    int member;
};
```

File "Parent.h" [edit]



gets copied

```
#include "Grandparent.h"
```

File "Child.c" [edit]



gets copied

```
#include "Grandparent.h"
#include "Parent.h"
```

Result [edit]



final “Child.c”

repetition :(
(compile error)

```
// From "Grandparent.h"
struct Foo {
    int member;
};
// From "Parent.h"
struct Foo {
    int member;
};
```

Src: https://en.wikipedia.org/wiki/Include_guard

Solution: include guards or `#pragma once`

If `#pragma once` is placed inside of a header file, that file is only included once.

```
#pragma once
// contents of the header
```

(Alternative: include guards)

```
#ifndef GRANDPARENT_H
#define GRANDPARENT_H

struct Foo {
    int member;
};

#endif // GRANDPARENT_H
```

Src: https://en.cppreference.com/cpp/preprocessor/impl

What to place in a header file?

(Best practice, CppCoreGuidelines) A header file must contain only:

- `#includes` of other header files (possibly with include guards)
- templates
- class definitions
- function declarations
- `extern` declarations
- `inline` function definitions
- `constexpr` definitions
- `const` definitions
- `using` alias definitions

Src: https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#sf-source-files

Types, structs and classes

C++ Type System

C++ is a **strongly, statically typed** language

=

Every object (variable, function argument and return value) has a type,

and the type **never changes**

Src: https://learn.microsoft.com/en-us/cpp/cpp/cpp-type-system-modern-cpp

Built-in Types

void

- represents the **empty set**
- can’t be used for variables
- mostly used for functions that don’t return a value

bool

- represents true or false (boolean)

Src: https://learn.microsoft.com/en-us/cpp/cpp/fundamental-types-cpp

Built-in Types

char

- represents a **single character**

float / double

- **floating point numbers** with different *ranges* (= *the minimum and maximum values they can hold*)

short / int / long / long long

- **integers** with different ranges

Src: https://learn.microsoft.com/en-us/cpp/cpp/fundamental-types-cpp

💡 Others

Useful to look into:

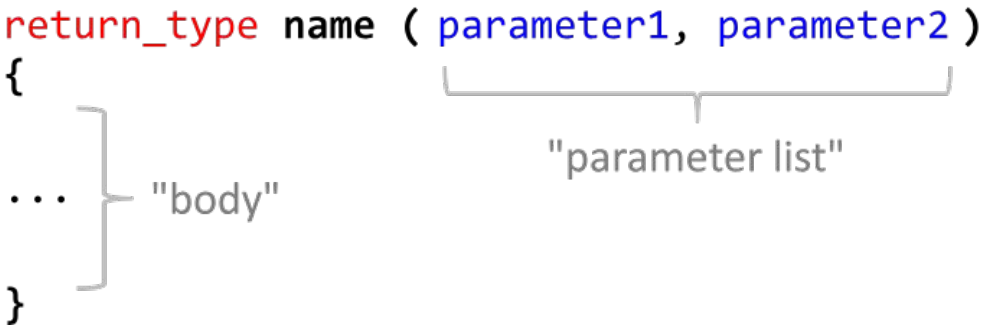
const

signed / unsigned

size_t

Src: https://learn.microsoft.com/en-us/cpp/cpp/fundamental-types-cpp

Functions



Src: https://cplusplus.com/doc/tutorial/functions/; https://hackingcpp.com/cpp/lang/function_basics

Tip: Memory management

The range of a type depends on the underlying *data model (= the system that the program is compiled for)*.

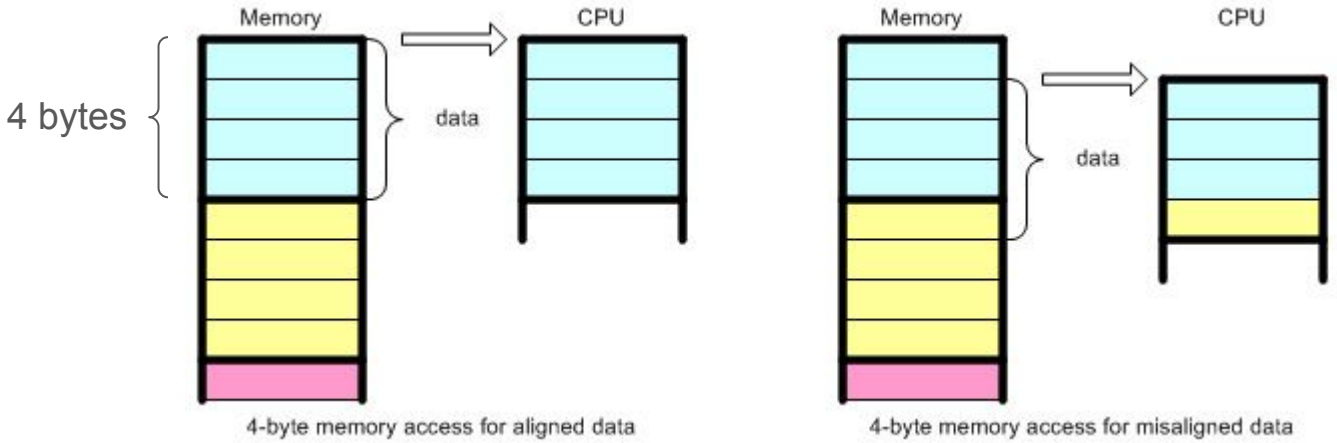
Types with smaller ranges (like `short`) need less memory, but can store a narrower range of numbers.

Src: https://en.cppreference.com/cpp/language/types

Memory alignment

CPU accesses memory in **byte chunks** of 2, 4, 8, 16, or 32 bytes.

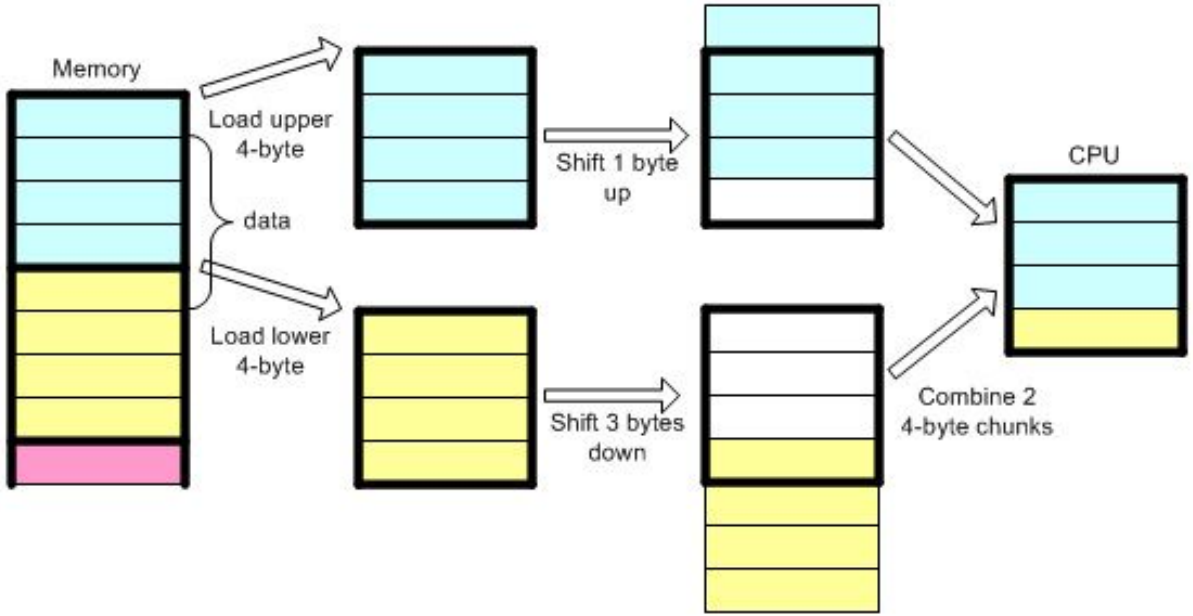
Aligned memory access = memory access where the data being accessed is n bytes long and the datum address is n-byte aligned. (Wikipedia)



Src: https://en.wikipedia.org/wiki/Data_structure_alignment
Img src: https://www.songho.ca/misc/alignment/dataalign.html

Memory alignment

Inefficient!

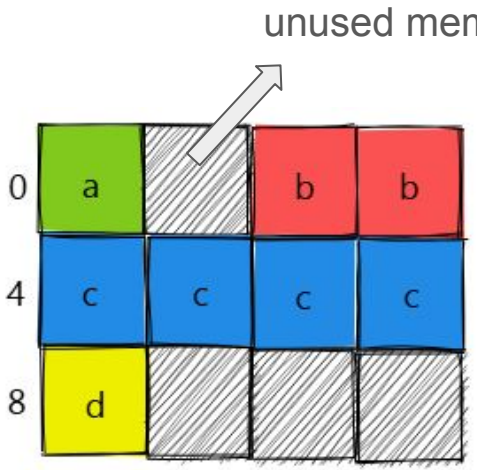


Img src: https://www.songho.ca/misc/alignment/dataalign.html

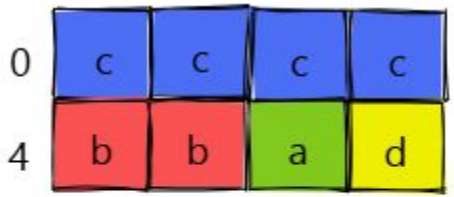
Struct

Struct = group of elements (member variables) whose storage is allocated in ordered sequence

```
struct foo {
    char a;
    uint16_t b;
    int32_t c;
    char d;
};
```




```
struct foo {
    int32_t c;
    uint16_t b;
    char a;
    char d;
};
```



To minimize the used space, declare the variables from biggest to smallest.

Src and img src: https://ncmiller.dev/memory-alignment.html

Class

Class = extension of the structure (originating in C) in C++

Src and img src: https://en.cppreference.com/cpp/language/classes

Member* functions

```
class Sample {
    void f(); // member func
};

void g(); // not a member func
```

Member functions = functions declared inside of a class

Accessing a member function happens through an object of the class:

```
MyClass obj;

obj.myFunction();
```

Src: https://www.ibm.com/docs/en/zos/3.2.0?topic=only-member-functions-c
* by member function we mean *non-static* member functions

Member* functions

Member functions have access to the other members (functions and variables) of the class.

```
class Wallet
{
     int balance = 50;
     bool canAfford(int amount)
     {
          return balance >= amount;
     }
};
```

Src: https://www.ibm.com/docs/en/zos/3.2.0?topic=only-member-functions-c
* by member function we mean *non-static* member functions

Constructors and destructors

The “life” of an object begins with its construction and ends with its destruction!

Src: https://www.ibm.com/docs/en/xl-c-and-cpp-aix/16.1.0?topic=only-overview-constructors-destructors-c

Constructors

Constructors often:

- Allocate memory
- Initialize variables

```
class X {
public:
  // Constructor for class X
  // has the same name as the class
  X();
};
```

The constructor of a class gets **automatically called** when an object of the class is created.

Src: https://www.ibm.com/docs/en/xl-c-and-cpp-aix/16.1.0?topic=only-constructors-c

Destructors

```
class X {
public:
  // Destructor for class X
  // is ~ + name_of_the_class
  ~X();
};
```

Destructors often:

- Deallocate memory
- Cleanup

The destructor of a class object is called when:

- the object passes out of scope or
- the object is explicitly deleted

Src: https://www.ibm.com/docs/en/xl-c-and-cpp-aix/16.1.0?topic=only-destructors-c

Access specifiers

Private

- Can only be used by other member functions of the **same class**

Protected

- Can be used by member functions of the **same class** and of **derived classes**

Public

- Can be used by **any function**

Src: https://en.cppreference.com/cpp/language/access
* this is a simplification, in case of *inheritance* please refer to the source for more information

Class VS Struct

They are the **same**, ***except***:

in a class:

- members are **private** by default
- base classes are **private** by default

in a struct:

- members are **public** by default
- base classes are **public** by default

Src: https://learn.microsoft.com/en-us/cpp/cpp/classes-and-structs-cpp

Writing classes

	Header File (.h)	**Source File (.cpp)**
Purpose	Defines the interface	Implements class functions
Contains	Function prototypes, class declarations, type definitions, macros, constants	Function implementations, executable code
Access	This is shared across source files	Is compiled into an object file
Example	`void someFunction();`	`void someFunction() {...};`

Img src: https://web.stanford.edu/class/cs106l/ (7. Classes)

Writing classes

```
1  // File: elevator.h
2
3  class Elevator {
4  private:
5    // Private member fields.
6    int floor;
7    int capacity;
8  public:
9    // Constructor.
10   Elevator(int capacity);
11   // Public member functions.
12   void request(int floor);
13 };
```

```
1  // File: elevator.cpp
2  #include "elevator.h"
3
4  Elevator::Elevator(int capacity) {
5    this->capacity = capacity;
6    floor = 0;
7  }
8
9  void Elevator::request(int floor) {
10   this->floor = floor;
11 }
```

Img src: https://web.stanford.edu/class/cs106l/ (7. Classes)

Passing by Value and by Reference

Pointers

Pointers = variables that **store memory addresses**

```
int a = 5;

// pointer that can store the address of a variable of type int:

int* myPtr;

// use & to get the address of a variable:

&a; // returns the address of a, e.g. 0x00000ABC

// save the address of a in the pointer:

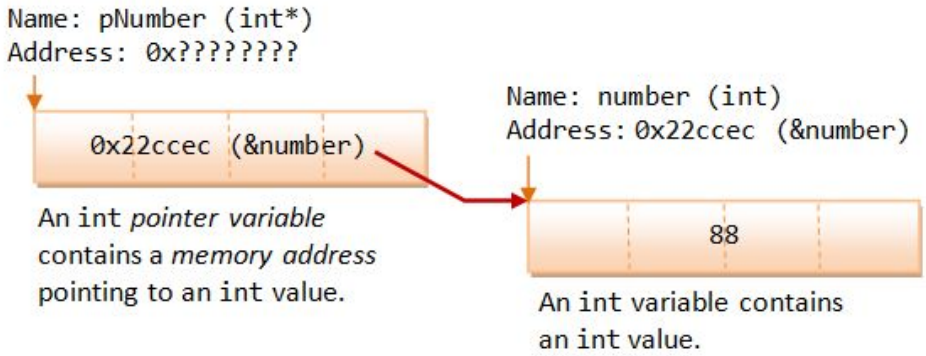
myPtr = &a;
```

Src: https://www3.ntu.edu.sg/home/ehchua/programming/cpp/cp4_PointerReference.html

Pointers

```
int number = 88;      // An int variable with a value
int * pNumber;        // Declare a pointer variable called pNumber pointing to an int (or int pointer)
pNumber = &number;    // Assign the address of the variable number to pointer pNumber

int * pAnother = &number; // Declare another int pointer and init to address of the variable number
```



Src and img src: https://www3.ntu.edu.sg/home/ehchua/programming/cpp/cp4_PointerReference.html

💡 Dereferencing operator

The **dereferencing operator** * helps us extract the value that is stored at the address in the pointer

```
int number = 88;
int * pNumber = &number;  // Declare and assign the address of variable number to pointer pNumber (0x22ccec)
cout << pNumber<< endl;   // Print the content of the pointer variable, which contain an address (0x22ccec)
cout << *pNumber << endl; // Print the value "pointed to" by the pointer, which is an int (88)
*pNumber = 99;            // Assign a value to where the pointer is pointed to, NOT to the pointer variable
cout << *pNumber << endl; // Print the new value "pointed to" by the pointer (99)
cout << number << endl;   // The value of variable number changes as well (99)
```

Src and img src: https://www3.ntu.edu.sg/home/ehchua/programming/cpp/cp4_PointerReference.html

References

Reference = an alias for an existing variable

A reference is similar to a pointer, **but**:

- once a reference is connected to a variable, it cannot reference another variable
- we don’t use the dereferencing operator with a reference

Src: https://www3.ntu.edu.sg/home/ehchua/programming/cpp/cp4_PointerReference.html

Pass-by-Value

Arguments of functions are passed by value by default, meaning that they are **copied** and passed to the function.

```
int square(int n) {
   cout <<  "In square(): " << &n << endl; // different address: 0x22ff00
   n *= n; // clone modified inside the function
   return n;
 }

int main() {
   int number = 8;
   cout <<  "In main(): " << &number << endl; // 0x22ff1c
   cout << number << endl;          // 8
   cout << square(number) << endl; // 64
   cout << number << endl;          // 8 - no change
}
```

Src: https://www3.ntu.edu.sg/home/ehchua/programming/cpp/cp4_PointerReference.html

Pass-by-Reference

When arguments are passed by reference, the **original copy** is passed directly, without cloning.

```
void square(int& n) {
   cout <<  "In square(): " << &n << endl; // same address: 0x22ff1c
   n *= n;
 }

int main() {
   int number = 8;
   cout <<  "In main(): " << &number << endl; // 0x22ff1c
   cout << number << endl;          // 8
   square(number);                  // call function
   cout << number << endl;          // 64 - value is updated
}
```

Src: https://www3.ntu.edu.sg/home/ehchua/programming/cpp/cp4_PointerReference.html

`const` Function Parameters

When we use `const`, the compiler makes sure the value is not modified inside of the function.

```
void square(const int& n) {
   n *= n; // error, value cannot be modified here!
 }

int main() {
   int number = 8;
   cout <<  "In main(): " << &number << endl;
   cout << number << endl;
   square(number); // error
}
```

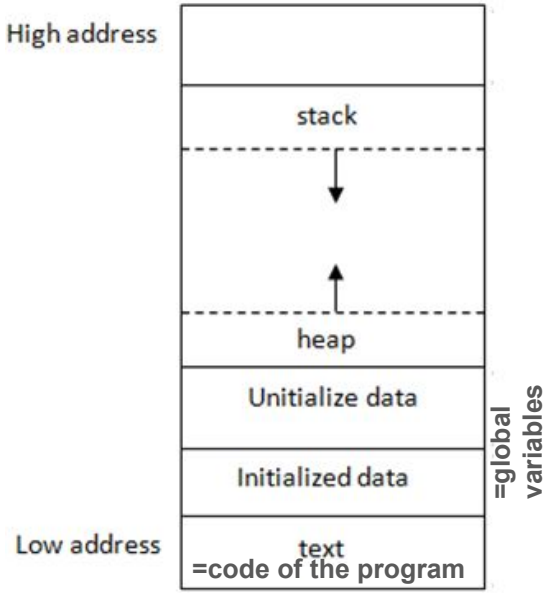
Src: https://www3.ntu.edu.sg/home/ehchua/programming/cpp/cp4_PointerReference.html; https://quuxplusone.github.io/blog/2022/01/23/dont-const-all-the-things/

Object Lifetime and Memory Management

Stack Vs Heap

Stack = place to store **local variables**

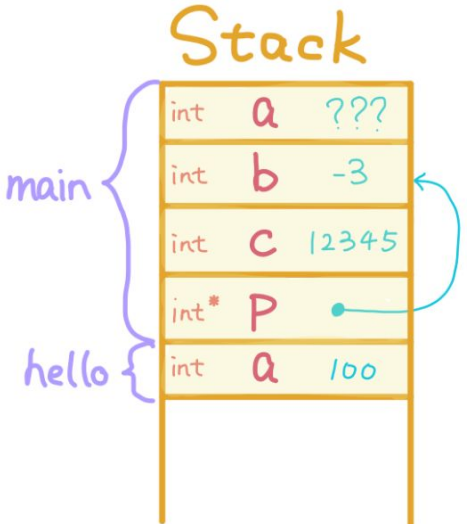
Heap = memory set aside for **dynamic allocation**



Src and img src: https://courses.grainger.illinois.edu/cs225/sp2020/resources/stack-heap/

Stack

- Every time a function is called, memory for it is allocated on the stack
- When the function returns, the memory is deallocated
- This is done ***automatically***

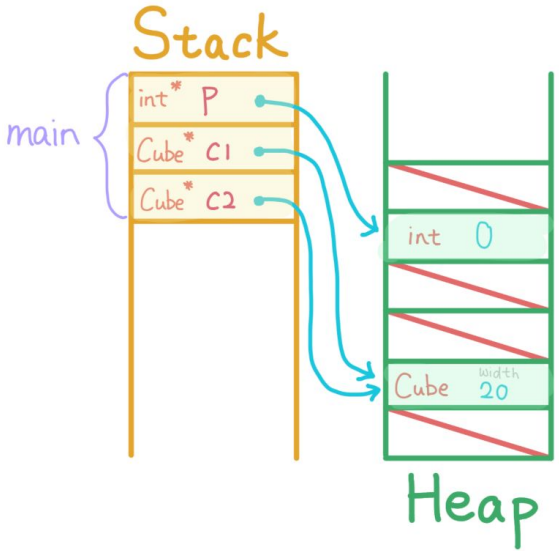



```c
int hello() {
  int a = 100;
  return a;
}
int main() {
  int a;
  int b = -3;
  int c = 12345;
  int *p = &b;
  int d = hello();
  return 0;
}
```

Src and img src: https://courses.grainger.illinois.edu/cs225/sp2020/resources/stack-heap/

Heap

- More permanent than stack memory
- Usually much bigger than what the stack allows for
- ***Explicitly allocated and freed by the developer***
- Not allocated consecutively




```
int main() {
  int *p = new int;
  Cube *c1 = new Cube();
  Cube *c2 = c1;
  c2->setLength( 10 );
  return 0;
}
```

Src and img src: https://courses.grainger.illinois.edu/cs225/sp2020/resources/stack-heap/

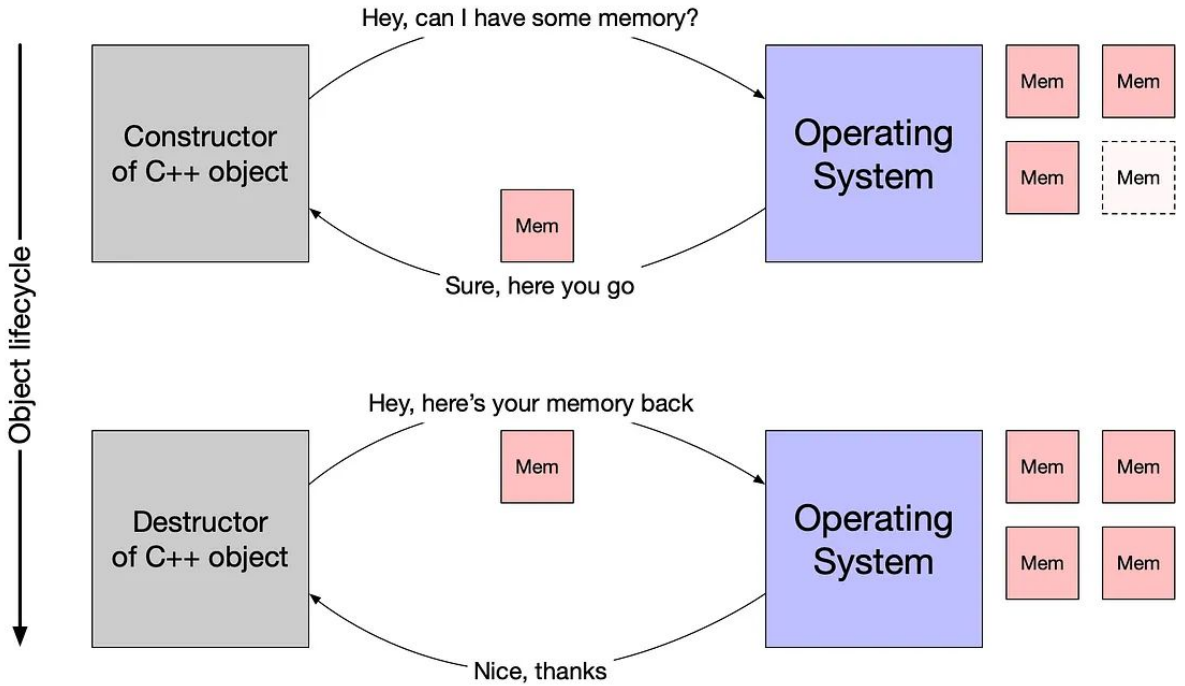
RAII (**R**esource **A**cquisition **I**s **I**nitialization)

RAII = *programming technique* which guarantees that:

- resources for the object (memory) are acquired **before use**
- the resources are available to all functions that **use** the object
- the resources are *properly freed* when the **use ends**

Src: https://en.cppreference.com/cpp/language/raii

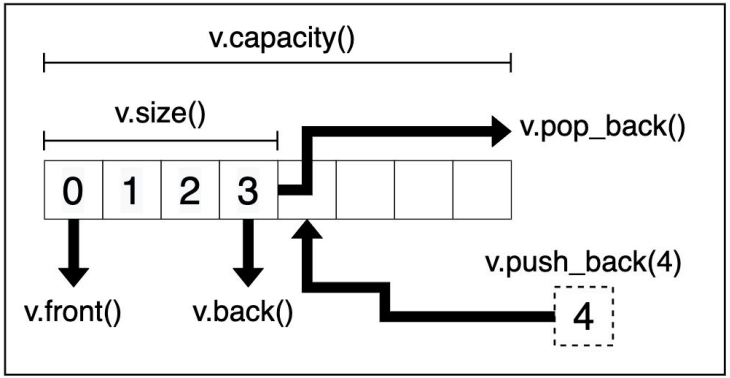
To summarize…



Img src: https://medium.com/swlh/what-is-raii-e016d00269f9

Storage Containers

`std::vector`



Vector = **dynamic** array

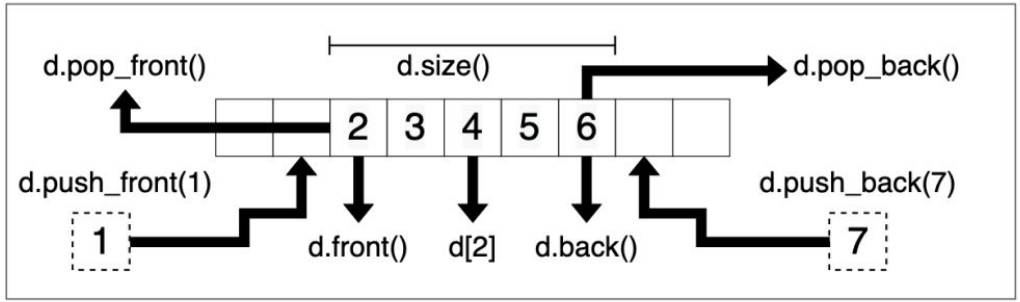
- Cells contiguously allocated in memory
- Random access is fast (**O(1)**)
- Insertion and removal at the end is fast (*usually* **O(1)**)
- Insertion/removal at random position are slower (**O(n)**)

Src: https://en.cppreference.com/cpp/container/vector
💡 Vector cheat sheet: https://users.cs.northwestern.edu/~riesbeck/programming/c++/stl-summary.html
Img src: https://clay-atlas.com/us/blog/2021/09/23/cpp-en-how-to-use-vector-stl/

`std::deque`

Deque = double-ended queue

- Elements are **not** stored contiguously in memory
- Fast access (**O(1)**)
- Fast insertions and deletions at the beginning and at the end (**O(1)**)
- Insertion/removal at specific position is slower (**O(n)**)

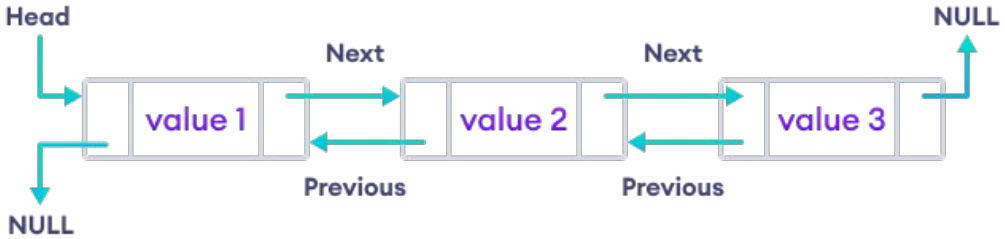


Src: https://en.cppreference.com/cpp/container/deque
💡 Deque cheat sheet: https://users.cs.northwestern.edu/~riesbeck/programming/c++/stl-summary.html
Img src: https://clay-atlas.com/us/blog/2021/09/28/cpp-en-how-to-use-deque-stl/

`std::list`

Doubly-linked list = each record has pointers to the previous and to the next element

- Non-contiguous memory allocation of the elements
- Access to an element requires traversal (**O(n)**)
- Easy to delete elements (but slower to find them)



Src: https://en.cppreference.com/cpp/container/list
💡 List cheat sheet: https://users.cs.northwestern.edu/~riesbeck/programming/c++/stl-summary.html
Img src: https://www.programiz.com/cpp-programming/list

`std::array`

Array = **fixed size** array

- Allocated on the stack or the heap, but **contiguously**
- Size of the array (number of elements) must be known *at compile time* (=should be provided before we compile the program)

Src: https://en.cppreference.com/cpp/container/array

Templates

Templates

“A class template is a cookie cutter for a description of how to build a family of classes that all look basically the same” (ISO C++)

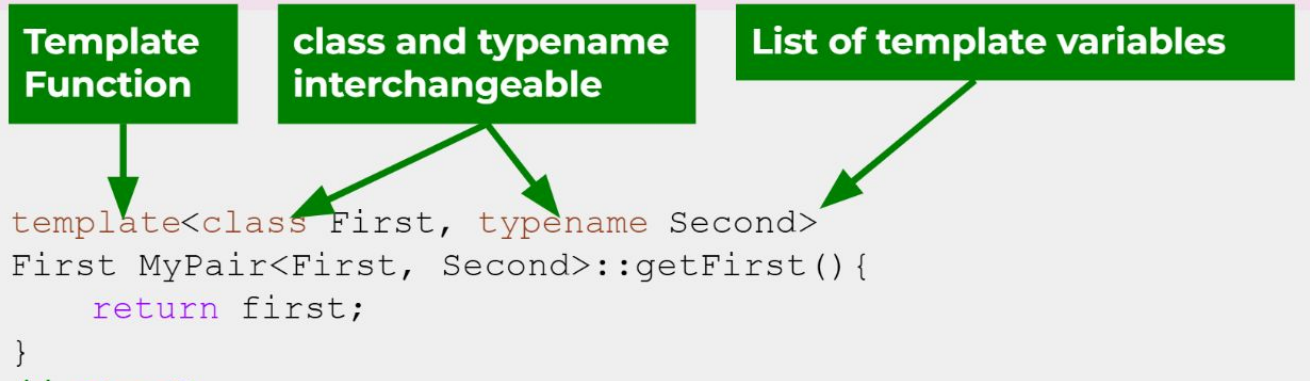
```
//mypair.h
template<typename First, typename Second> class MyPair {
    public:
        First getFirst();
        Second getSecond();

        void setFirst(First f);
        void setSecond(Second f);
    private:
        First first;
        Second second;
};
```

Generic! First and Second can be any type (including custom types) not just ints 😊

Img src: Stanford CS106L, Template Classes + Const Correctness (Spring 23)
Src: https://en.cppreference.com/cpp/language/templates; https://isocpp.org/wiki/faq/templates

Templates



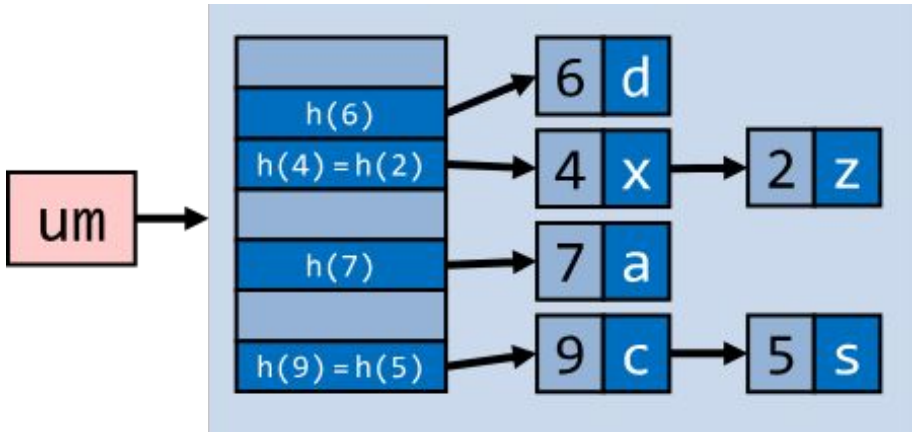
Img src: Stanford CS106L, Template Classes + Const Correctness (Spring 23)
Src: https://en.cppreference.com/cpp/language/templates; https://isocpp.org/wiki/faq/templates

Index (Associative) Containers

`std::unordered_map`

Unordered map = hash table (key-value pairs)

- Elements are stored based on hash value
- No guaranteed order of elements
- Insertion, lookup and removal are usually fast (**O(1)**)
- Can be slower (**O(n)**) depending on hash



Src: https://en.cppreference.com/cpp/container/unordered_map
Img src: https://hackingcpp.com/cpp/std/associative_containers

Useful links

Python to C++ Guide